

PROGETTO LAB II

Introduzione

Il progetto ha come obiettivo la realizzazione di un sistema multithread in C11 per la gestione di emergenze all'interno di una griglia bidimensionale. Il sistema è progettato per ricevere richieste di intervento tramite una coda di messaggi POSIX, assegnare in modo concorrente i soccorritori disponibili alle emergenze in base a priorità e distanza, e monitorare lo stato sia delle emergenze sia delle risorse assegnate.

Ogni unità di soccorso è rappresentata tramite un gemello digitale, che riflette il tipo, la posizione e lo stato operativo del soccorritore. Le emergenze vengono valutate in base a priorità e tempi di risposta massimi, e lo stato del sistema viene costantemente registrato in un log dettagliato.

Il sistema garantisce la corretta gestione della concorrenza attraverso mutex e condition variable, prevenendo situazioni di deadlock. Inoltre, è stato progettato per essere compatibile con Linux Ubuntu 24.04, utilizzando chiamate di sistema POSIX protette e controllate.

Architettura del sistema

Il sistema è progettato secondo un'architettura concorrente, che consente la gestione simultanea di più emergenze in una griglia bidimensionale. La comunicazione tra i componenti avviene tramite code POSIX e strutture dati condivise protette da meccanismi di sincronizzazione.

Componenti principali

- **Client**
 - Invia le richieste di emergenza alla coda POSIX.
 - Può inviare emergenze singole tramite linea di comando o multiple leggendo da file.
- **Server**

- Componente centrale del sistema.
- Prende il contenuto dei file .conf e lo distribuisce nelle adeguate strutture dati tramite vari **parser**
- Riceve le emergenze dalla coda POSIX e le inserisce in una coda interna prioritaria.
- Coordina l'assegnazione dei soccorritori e aggiorna gli stati delle emergenze.
- Utilizza thread dedicati per la gestione concorrente delle emergenze:
 - **Receiver thread:** legge le emergenze dalla coda POSIX.
 - **Dispatcher thread:** assegna i soccorritori alle emergenze in base a priorità, disponibilità e distanza.
- **Logger**
 - Registra tutte le operazioni significative del sistema in un file di log.
 - Eventi tracciati: ricezione emergenze, transizioni di stato di soccorritori e emergenze, assegnazioni e timeout.

Il progetto è suddiviso in più directory per mantenere separati i diversi moduli:

- **src/server/** – contiene il codice del server (server.c) , inclusa la logica per la gestione dei thread receiver e dispatcher (threads.c)
- **src/client/** – contiene il client che invia richieste di emergenza alla coda POSIX.
- **src/parser/** – contiene i moduli per il parsing dei file di configurazione (parse_rescuers.c, parse_emergency_types.c, parse_env.c).
- **src/utils/** – contiene moduli di supporto (logger, code prioritarie, gestione digital twins).
- **include/** – contiene gli header file condivisi tra i moduli. Nel file structures.h sono presenti tutte le strutture utilizzate dal sistema, nel file macro.h tutte le macro usate.

- **conf/** – file di configurazione (`rescues.conf`, `emergency_types.conf`, `env.conf`).
- **logs/** – file di log generati durante l'esecuzione.

Scelte progettuali

In questo paragrafo descrivo la principale decisione presa durante lo sviluppo del sistema, spiegando come e perché ho organizzato i vari componenti e thread. Il sistema si basa su due applicazioni separate: il client ed il server.

Client – Il client è stato progettato per permettere di inviare emergenze al server sia singolarmente tramite linea di comando, sia leggendo un file di input contenente più richieste. Per gestire l'invio concorrente delle emergenze ho deciso di usare thread separati: ogni emergenza viene inviata da un thread dedicato che attende il tempo di ritardo specificato prima di inserirla nella coda POSIX. Tutti i thread vengono attesi prima della chiusura del client, garantendo che tutte le emergenze siano inviate correttamente.

Server – Il server è stato progettato per gestire le emergenze in modo concorrente e modulare. All'avvio legge i file di configurazione tramite appositi parser per ottenere informazioni su soccorritori, tipi di emergenze e ambiente. Due thread principali, il Receiver e il Dispatcher, gestiscono rispettivamente la lettura dalla coda POSIX delle richieste ricevute dal client e l'assegnazione dei soccorritori tramite una coda interna prioritaria. Variabili di condizione e mutex garantiscono la sincronizzazione e la sicurezza della coda prioritaria, mentre il segnale SIGINT permette una terminazione ordinata.

Receiver – Il receiver_thread legge le emergenze dalla coda POSIX, ne verifica la validità e le inserisce nella coda interna a priorità per il dispatcher. Viene controllato il nome dell'emergenza, le coordinate e il timestamp e eventuali errori vengono loggati. Ogni emergenza valida viene associata ai digital twins dei soccorritori richiesti, permettendo di gestire lo

stato delle risorse in modo modulare. La terminazione sicura è garantita dalla variabile atomica `stop_flag` e dal corretto trattamento degli errori della coda.

Dispatcher – Il `dispatcher_thread` preleva le emergenze dalla coda interna a priorità e crea un thread separato per ciascuna, gestito da `handle_emergency`. Ogni thread lavora su una copia indipendente dell'emergenza e dei parametri, evitando conflitti di accesso e garantendo modularità. I thread vengono eseguiti in modalità `detach`, permettendo la gestione concorrente di più emergenze senza bloccare il server.

Handle_emergency – Il gestore dell'emergenza è stato progettato come thread separato per ogni emergenza ricevuta dal dispatcher. Questo approccio consente di gestire più emergenze contemporaneamente senza bloccare il server principale. Ogni thread emergenza verifica la disponibilità dei soccorritori necessari e confronta il tempo stimato di arrivo con il limite imposto dalla priorità dell'emergenza. Se le risorse non sono sufficienti, il thread emergenza attende usando una `condition variable`, evitando `busy-wait` e garantendo sincronizzazione sicura tra emergenze concorrenti. I soccorritori sono rappresentati tramite `digital twins`, ognuno con thread dedicati per simulare l'arrivo sulla scena e il ritorno alla base ed ognuno protetto da una `mutex` relativa all'emergenza, così che i thread soccorritori non accedano o modifichino contemporaneamente le stesse risorse, evitando `race condition`. Tutti i cambiamenti di stato sia delle emergenze, sia dei `digital twin` dei soccorritori, inclusi arrivi, gestione e completamento, sono registrati nel log. In caso di impossibilità di gestire l'emergenza in tempo, lo stato viene impostato a **TIMEOUT**; se invece le risorse nel sistema non sono sufficienti, lo stato diventa **CANCELED**.

Parser – Ogni parser legge un file di configurazione (`.conf`), ne valida i dati e costruisce le strutture dati interne (`rescuer_type_t`, `emergency_type_t`, `env_t`). In questo modo il server, una volta avviato,

lavora esclusivamente con strutture in memoria senza dover più interagire con i file.

Priority_queue – La coda è stata implementata come una lista collegata ordinata: quando arriva una nuova emergenza, viene inserita nel punto giusto della lista in base alla priorità. In questo modo l'estrazione (pop) restituisce sempre l'emergenza più urgente. Per garantire la correttezza in un ambiente multithread, ho protetto la coda con un mutex e ho usato una condition variable per coordinare i thread. Le operazioni di inserzione/rimozione vengono gestite da thread reciever/dispatcher.

Logger – Modulo che si occupa di scrivere in un logfile tutti gli eventi significativi del sistema. Ogni messaggio viene salvato con **timestamp, ID ed etichetta dell'evento**, così da avere una traccia chiara e ordinata dell'esecuzione.

Modifiche alle struct prestabilite:

- Ho aggiunto un campo occ_rescuers alla struct rescuer_type_t per contare il numero di soccorritori occupati per ogni tipo.
- Ho aggiunto un campo id alla struct emergency_t per fare in modo che ogni emergenza abbia il proprio id.
- Ho reso il campo rescuers_dt un array di puntatori dove ad ogni puntatore corrispondono i gemelli digitali di ogni soccorritore.

Istruzioni per compilare ed eseguire

Il progetto utilizza un Makefile per automatizzare la compilazione e la gestione degli eseguibili. Sono disponibili due target principali:

- `server` → compila il programma server
- `client` → compila il programma client

Per compilare entrambi: `make all`

Dopo la compilazione:

- Avvio server: `make server_run`
- Invio emergency tramite client: (fare da un altro terminale ripetuto a quello dove viene avviato il server)
 - Emergency singola: `./clientfile x y d -> x,y coordinate emergency, d delay emergency`
 - File di emergency: `./clientfile -f file.txt -> file.txt nome del file con le emergency`
- Terminazione server: `ctrl+c`

Pulizia:

- `Make clean`